

MP4: Crane Stream Processing

Architecture/Behavior:

The Crane stream processing system is built off of SDFS's utilities. We made use of Golang and its Plugin features to help distribute user-defined to workers real-time without having to recompile a portion of the code on the worker machines.

The architecture consists of a Master and all other machines being slaves to the master. The master will use the job details submitted to it from the SDFS cluster and the plugin to build out the topology and distribute tasks to each of the workers chosen to be part of the topology. Each worker/runner will pull their corresponding plugins for their tasks from the SDFS. The spout then starts streaming data, from a file stored in the SDFS, via TCP clients to its direct downstream nodes. Each of the workers maintaining bolts have a TCP servers to receive data from upstream and TCP clients to send processed data downstream. The workers maintaining sinks have TCP servers to receive incoming data and then they process the data into a file containing the results which is uploaded to SDFS following the completion of the job.

The behaviour of the system is as follows:

0. A <job_folder> and all plugins (topology, bolts, spouts) is created on the local machine within the SDFS/Crane cluster.
1. The same local machine submits <job_folder>/submit.txt to the Crane Cluster.
2. submit.txt will have .so files corresponding to every bolt and spout func in the system, as well as the topology.so, all contained in <job_folder>
3. When we call submit on the text file, the machine will PUT every node func into sdfs, and locally use the topology.so plugin to generate a topology tree, which we will then send to the master
4. The master will process the topology, distributing tasks amongst worker machines and starting up the spout (we assume a singular spout)
5. For each spout/bolt, the runner will pull down the associated .so file from sdfs, and get the associated plugin. We assume that every bolt plugin function takes input (topology.Tuple), which consists of an ID and Fields (map[string]string), and will return a reference to this same object, as well as a boolean indicating whether it has been passed (as opposed to filtered)
6. Each spout execute func should output a topology. Tuple as specified above

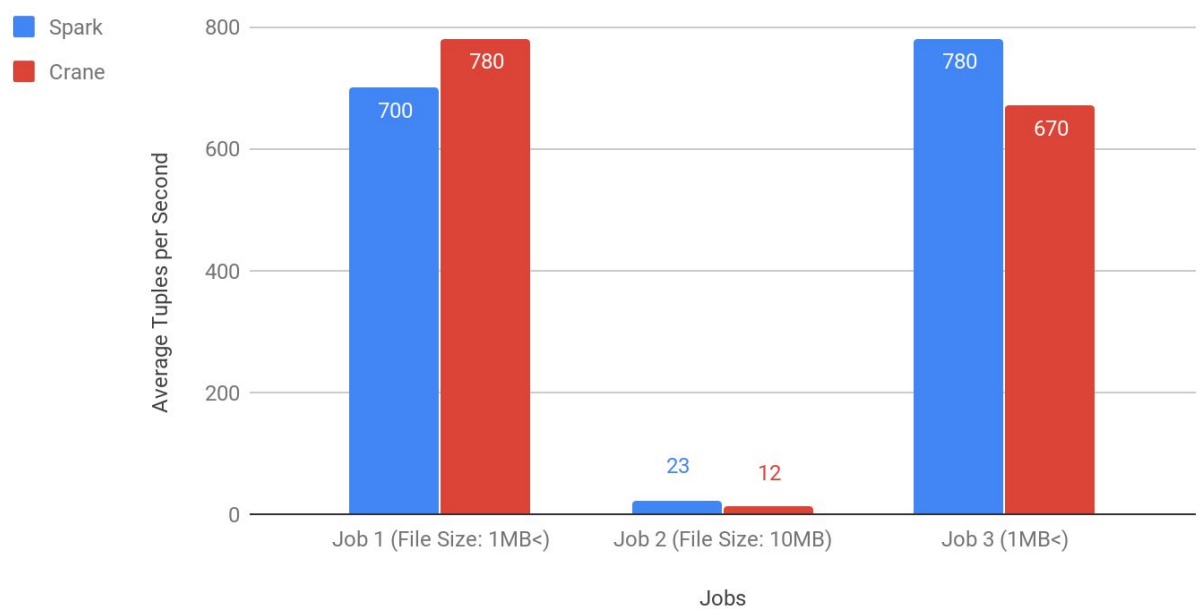
Programming Framework:

In tandem with our architecture, which consists of one master system, 9 other worker systems, and the submission system, we utilized gob serialization to transfer Tuples, Topologies, and Tuple Acks through the system. Each system ran either a worker or master routine, which allowed for multiple topologies to be handled in parallel (although we didn't experiment with

this functionality). The master kept track of nodes and workers so that failures were efficiently redistributed to other workers in the system.

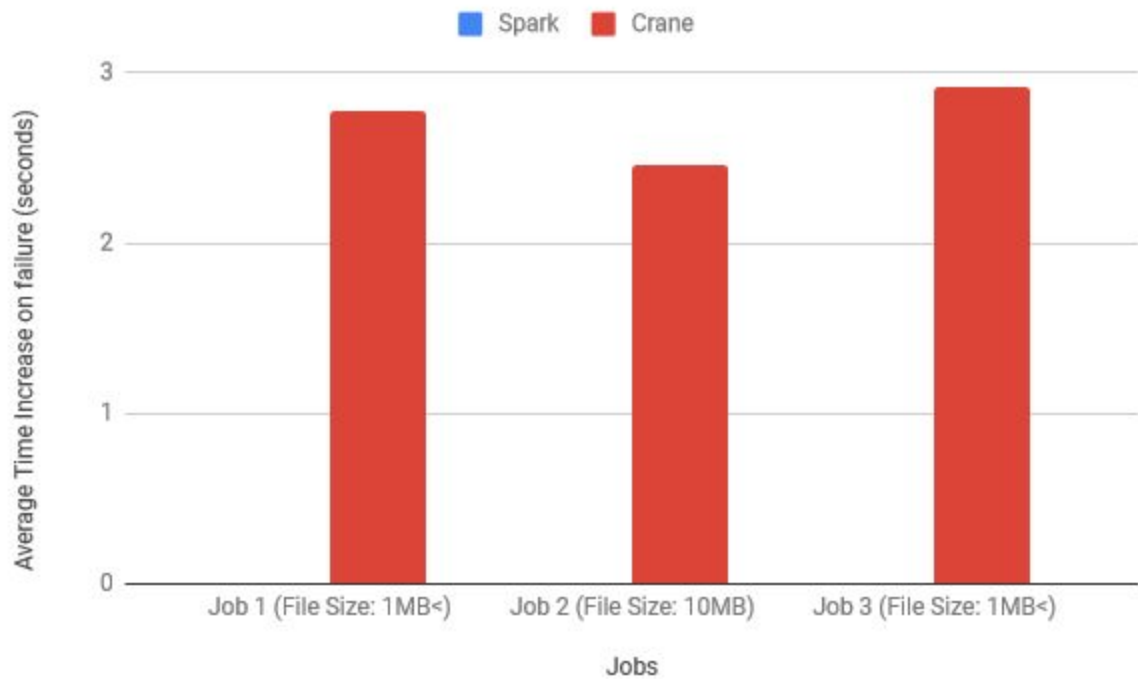
Results:

To evaluate the performance of our system we came up with a few metrics to compare against other systems, namely Spark. As our first metric, we decided to use tuples processed per second. We measured this through finding the total job runtime for each streaming service, and dividing it by the number of tuples sent from the spout.



On average Crane is slower on jobs with large datasets and faster on smaller datasets. A possible explanation for this is that Spark has a lot of overhead in terms of setting up jobs/topologies thus giving Crane an advantage when datasets are small.

We also used time increase on failure of a single machine as another metric. This is measured through first measuring the average time without any failures per job and then measuring the average time per job with one non-master failure a few seconds into the stream, and reporting the difference.



For Spark, a single failure seemed to not have an outsized impact on the time taken to complete a job. Crane was different as our method of failure handling had the job hold till the failed worker can be replaced.

Applications:

For Job 1, we decided to conduct a single topology that translated specific words to their associated scores, which we then conducted a filter on to identify usage of high-scoring words. On simple topologies such as Job 1, Crane was easily able to sift through the data, as there was not much overall work to be done. In addition, due to the fact that Crane operates only input programs that are acyclic in nature, the journey from spout to sink is very straightforward, in contrast with other streaming data processors that may have to handle cycled inputs.